Notice technique

Stores enrouleurs — XIAO ESP32-C3 / ESP32-S3 — programme v7.4

1. Présentation

Le programme commande un ou deux moteurs pas à pas 28BYJ-48 (drivers ULN2003) qui enroulent des stores. Il sert lui-même ses pages web : aucune application, aucun service Internet. Il tourne sur un XIAO ESP32-C3 ou un XIAO ESP32-S3, sans modification du code.

Principes de conception :

- **Les moteurs ne dépendent pas du réseau.** Un temporisateur matériel fait les pas toutes les 0,5 ms ; le Wi-Fi, la page web ou une écriture en mémoire ne peuvent pas ralentir un moteur.
- **Tout ce qui compte est mémorisé** (NVS) : calibration, position, réglages, compteurs. Une coupure de courant ne fait pas perdre la position (au plus 500 demi-pas).
- **Chaque carte est autonome.** Plusieurs cartes se découvrent entre elles et se pilotent depuis une page commune, mais chacune fonctionne seule si les autres sont éteintes.

2. Matériel et câblage

2.1 Composants

Élément	Caractéristiques utiles
XIAO ESP32-C3	RISC-V 160 MHz, Wi-Fi 2,4 GHz, USB natif
XIAO ESP32-S3	Double cœur 240 MHz, même brochage ; antenne externe fournie
28BYJ-48 5 V	Réduction environ 1/64 ; **4096 demi-pas par tour** de l'axe de sortie
ULN2003	4 transistors Darlington, une entrée par bobine
Alimentation 5 V	**1 A minimum** pour 2 moteurs (environ 200 mA par moteur en marche)

2.2 Raccordement

Driver	IN1	IN2	IN3	IN4	GPIO (C3)
Moteur 1	D2	D3	D4	D5	4, 5, 6, 7
Moteur 2	D1	D6	D7	D10	3, 21, 20, 10

- Le + 5 V des drivers vient de l'alimentation externe, jamais de la broche 5 V du XIAO quand l'USB est débranché.
- **Masse commune** obligatoire entre l'alimentation, les drivers et le XIAO.

- D6 (sortie série de démarrage) peut faire un petit à-coup du moteur 2 à la mise sous tension : sans conséquence.
- Broches libres : D0, D8, D9. Sur le C3 ce sont des broches de démarrage (GPIO2, 8, 9) : les réserver à des boutons avec résistance de rappel au + 3,3 V. Sur le S3 ce sont des broches ordinaires.

2.3 Compatibilité C3 / S3

Le programme utilise les noms de broches XIAO (D0…D10) : le paquet ESP32 les traduit vers les bons GPIO de chaque puce, le câblage est identique. Seul le type de carte change dans l'Arduino IDE. Un programme compilé pour l'une des puces ne fonctionne pas sur l'autre : ne jamais envoyer par OTA un fichier `.bin` compilé pour l'autre modèle.

3. Compilation et téléversement

Réglage (menu Outils)	Valeur
Type de carte	XIAO_ESP32C3 ou XIAO_ESP32S3 (paquet esp32 d'Espressif, 2.x ou 3.x)
USB CDC On Boot	**Enabled** (le moniteur série passe par l'USB, D6/D7 étant pris)
Partition Scheme	Un schéma avec deux zones OTA (celui par défaut convient)
Moniteur série	115200 bauds

Bibliothèques : toutes incluses dans le paquet ESP32 (WiFi, WebServer, ESPmDNS, Preferences, ArduinoOTA, Update, esp_timer). Rien à installer.

3.1 Fichier arduino_secrets.h

Constante	Rôle
`SECRET_SSID` / `SECRET_PASS`	Réseau Wi-Fi de la box
`SECRET_AP_SSID` / `SECRET_AP_PASS`	Réseau de secours créé par la carte (mot de passe : 8 caractères minimum). SSID différent sur chaque carte
`SECRET_OTA_PASS`	Mot de passe des mises à jour **et** du changement de mode. À défaut : `store-ota` (à changer)

4. Constantes de configuration

Les valeurs signalées par un drapeau rouge dans les commentaires du programme sont celles à adapter. Celles qui se règlent aussi sur la page web (sens, vitesse, nom, mode) ne servent qu'au **tout premier démarrage**, tant que rien n'a été enregistré.

4.1 Réseau et apparence

Constante	Défaut	Rôle
`HOSTNAME`	`store`	Nom réseau → `http://store.local`. Unique par carte
`PAGE_TITLE`	`Stores`	Titre de la page, nom du raccourci, titre du cadre sur `/tous`
`ICON_COLOR`	`#3d6fb6`	Couleur de fond de l'icône (format `#RRGGBB`)
`ALL_TITLE`	`Tous les stores`	Titre de la page `/tous` et de son raccourci
`FIXED_PEERS`	vide	Adresses IP des autres cartes, séparées par des virgules

Constante	Défaut	Rôle
AP_IP	192.168.5.1	Adresse de la carte en point d'accès de secours
STA_RETRY_MS	120000	Délai entre deux essais vers la box en mode secours (2 min)
PEER_SCAN_MS	30000	Délai entre deux recherches des autres cartes (30 s)

Éviter dans les titres les caractères `< > & "` et `\`.

4.2 Moteurs

Constante	Défaut	Rôle
MOTOR2_ENABLED	true	false : un seul moteur, page en pleine largeur
SYNC_DEFAULT	false	Mode au premier démarrage (true = synchronisés)
MOT1_INVERT / MOT2_INVERT	false	Sens au premier démarrage
MOT1_NAME / MOT2_NAME	Store 1 / Store 2	Noms par défaut (repris si on efface le nom)
MOT1_SIDE / MOT2_SIDE	Store Gauche / Store Droit	Titres des cadres sur la page
STEP_PERIOD_US	3000	Vitesse au premier démarrage (µs par demi-pas)
SAVE_EVERY_STEPS	500	Sauvegarde de la position en mouvement tous les 500 demi-pas

4.3 Vitesse, rampe, alignement

Constante	Défaut	Rôle
TICK_US	500	Période du temporisateur (0,5 ms) ; aussi le pas de réglage de la vitesse
SPEED_MIN_US	2000	Demi-pas le plus court (environ 7,3 tr/min)
SPEED_MAX_US	10000	Demi-pas le plus long (environ 1,5 tr/min)
RAMP_START_US	6000	Vitesse de départ et d'arrivée (environ 2,4 tr/min)
RAMP_STEPS	24	Demi-pas par cran de 0,5 ms (0 = pas de rampe)
JOG_MAX	4096	Alignement maximal en attente pour un moteur (1 tour)

Vitesse de l'axe de sortie : tr/min = 60 000 000 / (durée du demi-pas en µs × 4096). Exemple : 3000 µs → 4,9 tr/min. Plus le demi-pas est long, plus le moteur a de couple.

5. Fonctionnement des moteurs

5.1 Commande en demi-pas

Chaque moteur suit une séquence de 8 phases (une ou deux bobines alimentées). Un demi-pas = passage à la phase suivante (descente) ou précédente (montée). L'inversion de sens se fait en inversant le sens de parcours de la séquence. À l'arrêt, les 4 bobines sont coupées : le moteur ne chauffe pas et ne consomme rien ; le réducteur du 28BYJ-48 suffit à tenir le store.

5.2 Temporisateur

Un temporisateur `esp_timer` appelle `stepperTick()` toutes les 0,5 ms, dans la tâche dédiée aux temporisateurs. Chaque moteur compte les « tops » et fait un demi-pas tous les N tops (N = vitesse du moment). Si un top est retardé (écriture en flash), les tops manqués sont ignorés (`skip_unhandled_events`) : pas de rafale de pas au retour.

Échanges entre le temporisateur et la boucle principale : par variables `volatile`. La boucle donne les ordres (sens, cible) ; le temporisateur exécute, vérifie les butées et signale chaque arrêt (`stopEvent`). Les compteurs bruts ne sont écrits que par le temporisateur ; la boucle principale en reporte l'écart dans les totaux.

5.3 Position et butées

- Position en demi-pas : 0 = tout en haut, `travel` = tout en bas (course mesurée à la calibration).
- Avant chaque demi-pas, hors calibration : arrêt si la butée haute (position ≤ 0) ou basse (position ≥ course) est atteinte, ou si la cible d'un « Aller à » est atteinte.
- Pourcentage affiché = position / course, arrondi.
- Un moteur jamais calibré (course = 0) refuse Monter, Descendre et Aller à.

5.4 Rampe

Au départ, le moteur part à `RAMP_START_US` (6 ms) et gagne 0,5 ms tous les `RAMP_STEPS` (24) demi-pas jusqu'à la vitesse réglée. Avec les valeurs par défaut (12 tops → 6 tops) l'accélération dure 144 demi-pas, soit environ 1/28 de tour.

Avant l'arrivée (butée ou cible), la vitesse est limitée par la distance restante : limite = 12 − distance / 24 tops. Le moteur ralentit donc progressivement sur les 144 derniers demi-pas et arrive à 6 ms. Le ralentissement est appliqué immédiatement (sans risque), l'accélération cran par cran.

Pas de décélération pendant la calibration (l'arrivée n'est pas connue) ni sur « Stop » (arrêt immédiat, par sécurité). Si la vitesse réglée est déjà plus lente que 6 ms, il n'y a pas de rampe. Un changement de vitesse sur la page s'applique pendant le mouvement ; la rampe suit.

5.5 Calibration

1. **Démarrer** : arrêt du moteur, butées désactivées (mouvement libre dans les deux sens).
2. Monter jusqu'en haut, Stop, **Valider HAUT** : la position actuelle devient 0. Refusé si le moteur tourne.
3. Descendre jusqu'en bas, Stop, **Valider BAS** : la position actuelle devient la course totale. Refusé si le moteur tourne ou si la position n'est pas sous le haut.

La course et la position sont alors enregistrées et les butées réactivées. Pendant la calibration, « Aller à » est refusé et la position n'est pas sauvegardée. Une calibration inachevée est abandonnée par une mise à jour du programme ou un changement de mode.

5.6 Mode synchronisé

Le moteur 1 « mène » : son sens, sa position, sa cible, sa calibration, sa vitesse et sa rampe commandent les deux moteurs. À chaque demi-pas du moteur 1, le moteur 2 fait le même demi-pas, avec son propre réglage de sens (moteurs montés face à face). Le moteur 2 recopie la position et la course du moteur 1 après chaque commande. Toutes les commandes reçues, quel que soit le moteur visé, passent par le moteur 1. Un arrêt du moteur 1 arrête aussi le moteur 2.

Alignement : store à l'arrêt, un seul moteur tourne lentement (à 6 ms par demi-pas), sans changer la position affichée. Sert à remettre le store droit. Les déplacements demandés s'additionnent, dans la limite de ± 1 tour. Tout mouvement du store annule l'alignement en attente.

Changement de mode (protégé par `SECRET_OTA_PASS`) :

Passage	Effet sur la calibration
Indépendants → Synchronisés	Calibrations des deux moteurs effacées : une calibration commune est à refaire
Synchronisés → Indépendants	Chaque moteur garde la calibration commune

Les deux moteurs sont arrêtés et toute calibration en cours est abandonnée. Le mode est enregistré (clé `sync`).

5.7 Inversion du sens

Arrêt du moteur, puis effacement de sa calibration (en synchronisé : de la calibration commune), car le haut et le bas sont échangés.

6. Mémoire permanente (NVS)

Espace de noms `roller`. Clés par moteur (suffixe 1 ou 2) :

Clé	Type	Contenu	Écrite
`travel1`	long	Course en demi-pas (0 = non calibré)	Fin de calibration, effacement
`pos1`	long	Dernière position	À chaque arrêt et tous les 500 demi-pas en mouvement
`inv1`	bool	Sens inversé	Réglage depuis la page
`per1`	uint	Vitesse en µs par demi-pas	Réglage depuis la page
`name1`	texte	Nom du store (32 octets max.)	Réglage depuis la page
`steps1`	ulong	Total de demi-pas	À chaque arrêt
`moves1`	ulong	Total de mouvements	À chaque arrêt
`sync`	bool	Mode synchronisé	Changement de mode

Une position n'est écrite que si elle a changé. La NVS répartit l'usure sur ses pages : avec une écriture par arrêt et une tous les 500 demi-pas, la durée de vie de la flash n'est pas un sujet pour un store.

Reprise de la v3.1 : les anciennes clés communes `travelSteps` et `position` sont lues comme valeurs de départ si les nouvelles clés n'existent pas, puis réécrites sous les clés de chaque moteur au démarrage.

7. Réseau

7.1 Connexion à la box

- Mode client, veille Wi-Fi désactivée (page et OTA plus réactives).

- Puissance d'émission réduite à 8,5 dBm, réglée **après** `WiFi.begin()` : corrige les échecs de connexion du XIAO C3 (saturation). Sans utilité sur le S3, mais sans gêne.
- Attente de 15 s maximum. Ensuite, reconnexion automatique gérée par l'ESP32 en cas de perte.

7.2 Point d'accès de secours

Si la box ne répond pas dans les 15 s, la carte crée le réseau `SECRET_AP_SSID` à l'adresse 192.168.5.1 (mode point d'accès + client). Toutes les 2 minutes, elle retente la box — sauf si un téléphone est connecté au point d'accès, pour ne pas le perturber. Dès que la box répond, le point d'accès est coupé, mDNS relancé, et la carte reste sur la box. Cas typique : après une coupure de courant, la carte redémarre avant la box.

7.3 mDNS

La carte annonce :

Service	Rôle
`<HOSTNAME>.local`	Nom de la carte
`_http._tcp` port 80	Serveur web
`_arduino._tcp` port 3232	OTA : la carte apparaît dans Outils > Port de l'IDE
`_store._tcp` port 80	Signalement aux autres cartes (page `/tous`)

7.4 Découverte des autres cartes

Une tâche FreeRTOS séparée (`peerTask`) cherche le service `_store._tcp` 5 s après le démarrage puis toutes les 30 s. Une recherche mDNS bloque environ 3 s : dans sa propre tâche, elle ne ralentit ni la page ni les moteurs. La liste (16 cartes maximum, sans la carte elle-même) est protégée par un verrou (sémaphore). Si une recherche ne trouve rien, la liste précédente est gardée. Pas de recherche en mode secours ni pendant une mise à jour.

`FIXED_PEERS` complète la liste quand la box filtre le multicast (certaines box et réseaux maillés). Il peut être identique sur toutes les cartes : chacune ignore sa propre adresse.

7.5 Autorisation entre cartes (CORS)

La page `/tous`, servie par une carte, interroge directement les autres. Pour que le navigateur l'autorise, `/api/status` et `/api/cmd` renvoient l'en-tête `Access-Control-Allow-Origin: *`. Les autres adresses (réglages, mode, mise à jour) ne l'ont pas : elles ne se pilotent que depuis la page de leur propre carte.

8. Mise à jour du programme (OTA)

8.1 Depuis l'Arduino IDE

La carte apparaît dans Outils > Port sous son `HOSTNAME`. Mot de passe : `SECRET_OTA_PASS`. Port 3232.

8.2 Depuis le navigateur

1. Arduino IDE : Croquis > Exporter les binaires compilés.
2. Ouvrir `http://<carte>.local/update`, saisir le mot de passe OTA, choisir le fichier **.ino.bin** (pas « merged » ni « bootloader »), Envoyer.

3. Redémarrage automatique ; retour à la page principale au bout de 10 s.

8.3 Pendant une mise à jour

Les moteurs sont arrêtés, positions et compteurs sauvegardés, calibration en cours abandonnée, temporisateur suspendu (tout le processeur pour l'écriture). Les commandes sont refusées (« Mise à jour en cours »). En cas d'échec ou d'interruption, le fonctionnement normal reprend. Mauvais mot de passe sur `/update` : le fichier est ignoré, réponse 401.

9. Pages et API

9.1 Pages

Adresse	Contenu	Rafraîchissement
`/`	Commandes, calibration, réglages de la carte	500 ms
`/tous`	Toutes les cartes	1 s par carte (5 s si hors ligne)
`/info`	Informations et compteurs	2 s
`/update`	Mise à jour du programme	—
`/icon.png`	Icône 192 × 192 (fabriquée par le programme)	Gardée 1 jour
`/manifest.json`, `/manifest-tous.json`	Raccourcis du téléphone (ouvrent `/` ou `/tous`)	Gardés 1 jour

Le serveur web de l'ESP32 ne sert qu'une connexion à la fois. Les pages envoient donc **une seule requête à la fois par carte** (file d'attente), donnent la priorité aux commandes sur la lecture de l'état, et abandonnent une requête sans réponse au bout de 2 s (2,5 s sur `/tous`). Sur `/tous`, chaque carte a sa propre file : les cartes répondent en parallèle.

9.2 Commandes — /api/cmd

`/api/cmd?a=<action>&m=<moteur>[&p=<valeur>]` — `m` = 1 ou 2 pour un moteur, 0 pour les deux.

Action	Effet	Paramètre
`up` / `down`	Monter / descendre jusqu'à Stop ou la butée	—
`stop`	Arrêt immédiat, bobines coupées	—
`goto`	Aller à un pourcentage	`p` = 0 à 100
`calstart` / `caltop` / `calbot`	Étapes de calibration (`m` = 1 ou 2 obligatoire)	—
`jog`	Alignement d'un côté (synchronisé seulement)	`p` = demi-pas, négatif = monter

Réponses : 200 avec texte vide (accepté) ou avec un message (refus partiel quand `m` = 0) ; 409 avec le motif (refusé) ; 409 « Mise à jour en cours » pendant une OTA. En synchronisé, `m` est ramené à 1.

9.3 Réglages — /api/set

Requête	Effet
`?m=1&per=4000`	Vitesse (µs par demi-pas, bornée à 2000–10000, arrondie à 0,5 ms)

Requête	Effet
`?m=1&name=Salon`	Nom (caractères `" \ < > &` retirés ; vide = nom par défaut)
`?m=1&inv=1`	Sens inversé (efface la calibration)
POST `sync=1&pw=…`	Mode : 0 = indépendants, 1 = synchronisés. 401 si mauvais mot de passe

La page envoie le changement de mode en POST : le mot de passe n'apparaît ni dans l'adresse ni dans l'historique.

9.4 État — /api/status (JSON)

Champs généraux : `title`, `host`, `ota`, `ap`, `sync`, `rssi`, `pmin`, `pmax`, `ptick`, `spr`. Tableau `m`, un bloc par moteur : `name`, `side`, `pct`, `step` (position), `travel`, `dir` (−1 montée, 0 arrêt, +1 descente), `cal`, `top`, `ready` (calibré), `inv`, `per`, `jog`.

9.5 Autres

Adresse	Contenu
`/api/info`	Version, date de compilation, durée de fonctionnement, cause du démarrage, mode, processeur, réseau, IP, RSSI, MAC, mémoire, compteurs
`/api/info?reset=1`	Remise à zéro des compteurs du moteur 1 (2 pour le moteur 2)
`/api/peers`	`self` (IP de la carte), `fixed` (`FIXED_PEERS`), `peers` (cartes trouvées : `host`, `ip`)

10. Compteurs d'usure

Par moteur : **mouvements** (un par démarrage), **demi-pas** (affichés en tours : demi-pas / 4096), **cycles complets** = demi-pas / (2 × course), arrondi à l'entier inférieur. En synchronisé, le moteur 2 compte les mêmes pas que le moteur 1, plus ses alignements. Écrits en NVS à chaque arrêt ; remise à zéro depuis la page `/info`.

11. Dépannage

Symptôme	Cause probable	Solution
Rien sur le moniteur série	USB CDC On Boot désactivé	Outils > USB CDC On Boot > Enabled
`store.local` introuvable	mDNS non résolu (fréquent sous Android)	Utiliser l'adresse IP, réservée dans la box
La carte crée son réseau de secours	Box introuvable au démarrage	Attendre : nouvel essai toutes les 2 min ; vérifier SSID / mot de passe
« Carte injoignable » en bas de page	Carte éteinte, hors portée ou redémarrage	Vérifier l'alimentation et le signal (page Informations)
Signal faible (en dessous de −75 dBm)	Distance, puissance réduite à 8,5 dBm	Rapprocher la carte ; sur S3, brancher l'antenne
Autres cartes absentes de `/tous`	Box qui filtre le multicast, `HOSTNAME` identiques	`HOSTNAME` uniques ; sinon `FIXED_PEERS`
Moteur qui vibre sans tourner	Couple insuffisant, alimentation trop faible	Ralentir (curseur vers « Lent ») ; alimentation 1 A

Symptôme	Cause probable	Solution
Moteur qui tourne à l'envers	Montage du moteur	« Inverser le sens » puis recalibrer
« non calibré »	Course nulle	Faire la calibration
Store de travers (synchronisé)	Glissement d'un côté	Carte « Alignement des deux côtés »
Mise à jour refusée	Mauvais mot de passe ou mauvais fichier	`SECRET_OTA_PASS` ; fichier `.ino.bin` compilé pour ce modèle
Redémarrages (« baisse de tension »)	Alimentation insuffisante	Alimentation séparée pour les moteurs, masse commune

La page **Informations** donne la cause du dernier démarrage (mise sous tension, mise à jour, plantage, chien de garde, baisse de tension…) : premier réflexe en cas de comportement anormal.

12. Limites connues

- Pas de capteur de fin de course : la position est déduite des pas. Un store bloqué mécaniquement ou forcé à la main décale la position ; refaire alors la calibration.
- Pas d'authentification sur les commandes : toute personne connectée au Wi-Fi de la maison peut piloter les stores. Seuls la mise à jour et le changement de mode sont protégés.
- Pages en HTTP simple (pas de HTTPS), adaptées à un réseau domestique.
- `/tous` garde les cartes vues pendant la session (en grisé si elles ne répondent plus). Une carte retirée disparaît après rechargement de la page, dès que les autres cartes ont refait une recherche qui ne la trouve plus.